

Vesting Wallet Audit



July 9, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Vesting Wallet Primitive	5
Linear Curve	6
Security Model and Trust Assumptions	7
Low Severity	9
L-01 Coins Sent to a Wrapped Wallet's Address Are Unclaimable Until Unwrap	9
Notes & Additional Information	10
N-01 Redundant Recomputation of the Wallet Identifier in release	10
N-02 Incorrect Documentation	10
N-03 Missing Documentation	11
N-04 Style Guide Deviations	12
N-05 Permissionless Release Can Fragment Payouts Into Many Small Coins	13
N-06 Released Event Omits Payout Coin ID	13
Client Reported	14
CR-01 Wallets With an Object-Address Beneficiary Cannot Be Torn Down	14
Conclusion	16
Appendix	18
Issue Classification	18

Summary

Type	Library	Total Issues	8 (8 resolved)
Timeline	From 2026-06-23 To 2026-06-25	Critical Severity Issues	0 (0 resolved)
Languages	Move	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	1 (1 resolved)
		Notes & Additional Information	6 (6 resolved)
		Client Reported Issues	1 (1 resolved)

Scope

OpenZeppelin performed an audit of the [OpenZeppelin/contracts-sui](#) repository at commit [8faef63](#).

In scope were the following files:

```
contracts
├── finance
│   └── sources
│       ├── vesting_wallet.move
│       └── vesting_wallet_linear.move
```

System Overview

The Vesting Wallet is a curve-agnostic primitive in the `openzeppelin_finance` package that locks a single coin type for one beneficiary and releases it over time according to a vesting schedule. The design separates two concerns: a wallet object that custodies funds and enforces release accounting, and a schedule curve that defines how much has vested at any point in time. The curve is not built into the wallet; it is supplied by a separate module through type parameters, so a single primitive can back a linear schedule or any custom schedule a downstream module defines. The package ships one such curve, a stepped (tranche) schedule that vests in equal tranches, one per period, after an optional cliff (reducing to continuous linear-with-cliff vesting in the one-tranche-per-millisecond limit). This built-in linear curve is the one most integrators use directly, and it serves as the reference curve for new schedules. The primitive itself is expected to be used directly by two other audiences: authors writing a custom curve on its release-accounting core, and protocols that drive a wallet curve-agnostically, such as wrapping or gating release without committing to a schedule.

The engagement covered two modules, `vesting_wallet` and `vesting_wallet_linear`. The three modules under `examples/vesting_wallet` (`example_pausable_grant`, `example_splitter`, and `example_vesting_quadratic`) are out of scope; they were reviewed only as references for the intended integration patterns and not as audited code, and each is explicitly marked as an unaudited illustration that is not production ready.

A wallet is parameterized by three types fixed at construction: a schedule witness, the schedule parameters, and the coin type. Because struct fields are module-private in Move, only the module that declares the witness and parameter types can build a wallet of that type or attest how much it has vested, which binds each wallet to exactly one curve at the type level rather than through a runtime check. Funding and release are permissionless, attestation and teardown finalization are gated by the curve's witness, and the consumer chooses whether the wallet is a shared or owned object after construction.

Vesting Wallet Primitive

The `vesting_wallet` module is the stateful, curve-agnostic core on which curves and curve-agnostic protocols build. A `VestingWallet` object holds a balance, the fixed beneficiary address, the cumulative amount already released, and the curve's schedule

parameters. It never interprets the schedule; it only enforces fund conservation and the accounting that no payout exceeds what has vested or what the balance can cover. Its main flows are:

- Construction through `new`, which records the beneficiary and parameters and returns the wallet for the consumer to share or transfer.
- Funding through `deposit` and the address-targeted `receive_and_deposit`, both permissionless.
- Attestation through `mint_vested_amount`, gated by the curve witness and stamped with the wallet's id.
- Release through `release`, which pays the newly vested portion to the beneficiary.
- Teardown through the permissionless `destroy_empty`, which returns a receipt that only the curve can finalize through `consume_receipt`.

Linear Curve

The `vesting_wallet_linear` module is the built-in curve that most integrators would use directly, and the template that custom curves follow. It is a stateless library that declares the `Linear` witness and the `Params` struct, evaluates the stepped (tranche) schedule against the clock, and mints attestations against a wallet of its type. It re-derives the wallet's total from the balance plus the released amount on every call, so deposits made after the schedule starts participate at the current proportion. Its main flows are:

- Schedule construction through `params`, with `new` and `create_and_share` as sugar over the primitive's constructor.
- Evaluation through `vested_amount`, which reads the clock and the wallet's total and mints a `VestedAmount` for the `Linear` curve.
- Release and its preview through `release` and `releasable`, which delegate to the primitive.
- Teardown through `destroy`, which consumes the receipt once the schedule has ended and the caller is the beneficiary.

Security Model and Trust Assumptions

The vesting wallet enforces its guarantees through Move's type system and capability model rather than runtime access control. The primitive itself enforces fund conservation and the cap that no release exceeds the funded balance, while the correctness of the vesting schedule is delegated entirely to the curve module that holds the witness. The system's safety therefore rests on the honesty of curve modules, the integrity of the underlying Sui object and transfer model, and correct configuration by the integrator.

The system relies on the following trust assumptions:

- The curve module is trusted to attest honest amounts. The wallet never re-derives the schedule and accepts whatever amount the witness-holding curve mints, so a curve is expected to keep its attested total monotonically non-decreasing and bounded by the balance plus the released amount. The wallet-id stamp confines any deviation to the curve's own wallets, and the primitive still caps every payout at the wallet's balance, so a curve can affect only the wallets it created and only up to their funded balance.
- The integrator is trusted to choose the wallet topology deliberately. A shared wallet exposes permissionless release to anyone, while an owned wallet couples funding, release, and teardown to the holder, who must produce the wallet by reference or value.
- The beneficiary address may be a signer account or an object address. An object-address beneficiary receives each release as a transfer-to-object payout that the owning object is trusted to actively claim and to keep claimable.
- Funding and release are permissionless and route funds only to the beneficiary recorded at construction. The beneficiary is data rather than a capability and is fixed for the wallet's lifetime.
- The system depends on the underlying Sui network for the integrity of the clock used to evaluate schedules and for the transfer-to-object and receiving model used to fund wallets and to pay object beneficiaries.

The system defines the following privileged roles:

- Curve module (holder of the witness): the only party that can mint a `VestedAmount` for a wallet of its type through `mint_vested_amount` and the only party that can finalize teardown by consuming the receipt through `consume_receipt`. For the linear curve this authority is exercised through `vested_amount` and `destroy`.

- Beneficiary: the fixed recipient of every [release](#) and, in the built-in linear curve, the only address permitted to finalize [destroy](#) once the schedule has ended.
- Wallet holder (owned topology): in owned mode, the sole party able to pass the wallet by reference or value, and therefore the only one able to fund it, trigger releases, or tear it down.

Low Severity

L-01 Coins Sent to a Wrapped Wallet's Address Are Unclaimable Until Unwrap

A `VestingWallet` accepts address-targeted funding: a payer `public_transfer`s a `Coin<C>` to the wallet's object address, and the coin is later drawn into the balance by `receive_and_deposit`, which requires `&mut` access because it calls `transfer::public_receive(&mut wallet.id, ..)`. The [wrapper pattern the primitive documents](#) for a curve-agnostic third party that nests the wallet exposes only an immutable `&inner` and keeps `&mut inner` private, illustrating a `release` passthrough that reaches that private `&mut` internally.

A wrapper built to this guidance re-exposes whatever passthroughs it chooses, but the guidance illustrates only `release`. A wrapper that does not also add a `receive_and_deposit` passthrough leaves no way to draw in a coin parked at the inner wallet's address, because the claim needs the `&mut inner` the wrapper holds privately. Anyone can still `public_transfer` a `Coin<C>` to that address, so a coin funded this way while the wallet is wrapped strands until the wallet is unwrapped, after which `&mut` is restored and `receive_and_deposit` can claim it. The funds are recoverable and never lost, so the impact is a temporary liveness and usability hazard, but the window is a property of the core receive model combined with the recommended wrapping, not of any particular wrapper.

Consider amending the wrapper-pattern guidance to cover address-targeted funding, either by advising wrappers to add a `receive_and_deposit` passthrough alongside the `release` one or by documenting the strand window in that guidance and in the `receive_and_deposit` note, so an integrator who funds a wrapped wallet by address knows the coin stays unclaimable until the wallet is unwrapped.

Update: Resolved in [pull request #446](#). The team stated:

| Updated documentation to clarify

Notes & Additional Information

N-01 Redundant Recomputation of the Wallet Identifier in `release`

The `release` function [asserts that the attestation's `wallet\_id` equals `object::id\(wallet\)`](#) before paying the beneficiary. When emitting the `Released` event it nonetheless [re-derives `object::id\(wallet\)`](#) instead of reusing the `wallet_id` the assertion just validated. The derivation is deterministic and the wallet reference is unchanged between the two sites, so the emitted identifier is always correct and no balances or accounting are affected. The recomputation is therefore redundant and inconsistent with [`destroy\_empty`](#), which derives the identifier once and reuses it for the `Destroyed` event.

Consider emitting the `Released` event from the already-validated `wallet_id` binding instead of recomputing `object::id(wallet)`. Because `ID` has the `copy` ability, dereferencing the binding reproduces the value without re-deriving it from the wallet, and aligns `release` with the derive-once pattern used in `destroy_empty`.

Update: Resolved in [pull request #447](#). The team stated:

| *Resolved*

N-02 Incorrect Documentation

Several doc comments in the `vesting_wallet_linear` and `vesting_wallet` modules describe behavior that the implementation does not exhibit:

- The [linear curve doc](#) states that the mid-schedule cumulative total `total * k / steps` is "computed with a u128 intermediate". The value is produced by `openzeppelin_math::u64::mul_div`, called as `mul_div(total, elapsed_steps, steps, rounding::down())` [here](#). `u64::mul_div` delegates to a package [macro that promotes all operands to u256](#), as the macro's own documentation states, so no `u128` value exists on this path. A `u128` product would mathematically suffice (since `u64 * u64 < 2^128`), but the code does not use one.

- The [curve-contract doc](#) states that a curve violating monotonicity or the `balance + released` upper bound "makes release abort before any state mutation". This holds for upper-bound violations (`EInsufficientBalance`, asserted [here](#)) and for monotonicity violations only when the attested cumulative drops below `released` (`EVestedBelowReleased`, asserted [here](#)). A non-monotone dip that stays at or above `released` does not abort: `release` computes `vested - released` and silently pays the smaller increment.

Consider rewording the linear curve documentation to state that the product is evaluated in a wider integer (`u256`) before the floor division. Consider also tightening the curve-contract documentation to the actual conditions: a regression below `released` aborts with `EVestedBelowReleased`, exceeding `balance + released` aborts with `EInsufficientBalance`, and an in-range regression passes silently and releases the smaller increment.

Update: Resolved in [pull request #449](#). The team stated:

| *Applied suggestions*

N-03 Missing Documentation

The `vesting_wallet` documentation omits information that integrators rely on:

- The [#### Aborts section of receive_and_deposit](#) lists only `EBalanceOverflow`, but the function [calls transfer::public_receive](#), a thin wrapper over the native `receive_impl` that can abort. The realistic abort is `EUnableToReceiveObject` (`sui::transfer` code 3): it fires when the `Receiving<Coin<C>>` ticket is no longer receivable through the wallet, that is, the coin was already claimed by an earlier or concurrent transaction (a stale-version double-receive race), the object was wrapped or transferred away or is absent at that version, or the parent passed in is not its owner. `EReceivingObjectTypeMismatch` (code 2) is effectively unreachable because the input is typed `Coin<C>` at the Move boundary. A `Receiving` cannot be forged in Move (it is a declared transaction input), so the trigger is a genuinely stale or racing ticket rather than arbitrary attacker input, and the impact is documentation consistency only with no behavior change.
- The [Topologies section](#) recommends Shared and describes the Owned mechanism in its Owned bullet but never names the holder-withholding risk. In owned topology, `release`, `deposit`, `receive_and_deposit`, and `destroy_empty` all require `&mut` or by-value access that only the owner can produce. An owner who is not the

beneficiary and is uncooperative can withhold every payout, and the beneficiary has no on-chain path to force one.

Consider documenting the `public_receive` abort in the `receive_and_deposit` `#### Aborts` section, and adding a sentence to the Owned-topology bullet that names the withholding and liveness risk and ties the recommended Shared topology to mitigating it (Shared `release` is permissionless).

Update: Resolved in [pull request #450](#). The team stated:

| *Suggestions applied*

N-04 Style Guide Deviations

The linear curve module contains two minor deviations from the style guide:

- The call to `vesting_wallet::consume_receipt(receipt, Linear {})` is module-qualified rather than written in receiver syntax. The first parameter of `consume_receipt` is `receipt: DestroyReceipt<S, P>` by value, so the `receiver syntax rule`, which prefers `obj.fn(...)` whenever the function's first parameter is the object, applies in this situation. The [quadratic example already uses](#) `receipt.consume_receipt(Quadratic {})` for the identical call, so the linear module is internally inconsistent. The fix is `let (beneficiary, params) = receipt.consume_receipt(Linear {});`.
- The `SAFETY comment` is missing a space, so the word `and` runs directly into the following `start_ms` reference with no separating space. It is an ordinary `//` comment, not a `///` doc-comment, so nothing is rendered, and it is purely a source-readability typo.

Consider rewriting the `consume_receipt` call in receiver syntax to match the quadratic example and the style guide, and inserting the missing space in the `SAFETY` comment.

Update: Resolved in [pull request #448](#). The team stated:

| *Resolved*

N-05 Permissionless Release Can Fragment Payouts Into Many Small Coins

On the recommended shared topology, `release` is permissionless and pays the amount vested since the last call to the beneficiary as a fresh `Coin<C>` via `public_transfer`. Because it pays `vested - released` and reads `released` fresh, a repeat call at the same clock value vests nothing and is a no-op, so each call yields at most one coin, and only once the clock has advanced to a newly vested tranche. Anyone, not only the beneficiary, can make these calls, so a third party can split a wallet's payout into many small coins by calling `release` repeatedly as the schedule progresses.

The effect is bounded and costly to force. Even with the finest schedule (`params_continuous`, one tranche per millisecond), the coin count is capped by the number of distinct clock values over the vesting window, so reaching thousands of coins means submitting a fresh transaction at nearly every clock tick across the whole duration, paying gas throughout for no gain. Totals are always preserved and no funds are lost; the only consequence is many small `Coin<C>` objects for the beneficiary to hold, or, for an object beneficiary, many `Receiving`s to process.

Consider noting this fragmentation property in the `release` or shared-topology documentation, so an integrator that points a wallet at an object beneficiary plans for the possibility of many small payouts rather than a few large ones.

Update: Resolved in [pull request #451](#). The team stated:

| *Resolved*

N-06 Released Event Omits Payout Coin ID

The vesting wallet supports object-address beneficiaries: the [recipient can be a consumer-owned object](#). When the beneficiary is an object, a payout is not delivered into a signer's inventory but [queued at the object's address](#) as a `Receiving<Coin<C>>`. It must later be claimed by supplying it to `transfer::public_receive`. That input references the specific payout coin object by its id, which the caller must know when building the claim transaction.

The `Released event`, however, records only `wallet_id`, `beneficiary`, and `amount`, not the payout coin's object id. An off-chain process tracking an object beneficiary therefore cannot correlate a `Released` event with the specific queued `Receiving<Coin<C>>` it produced, and must instead enumerate the beneficiary's owned coins to discover and claim pending payouts. Proving that every release has been claimed is correspondingly harder, and

this gap widens under permissionless release, which lets any third party fragment payouts into more queued receivings.

Consider including the payout coin's id in `Released`, for example by emitting `object::id(&coin)` before the `public_transfer`, so off-chain consumers can correlate a specific `Released` event with a specific pending `Receiving<Coin<C>>`.

Update: Resolved in [pull request #452](#). The team stated:

| *Resolved*

Client Reported

CR-01 Wallets With an Object-Address Beneficiary Cannot Be Torn Down

The vesting wallet supports [object-address beneficiaries](#), and teardown is split into two halves: the permissionless `destroy_empty` consumes a drained wallet and returns a `DestroyReceipt`, which the curve module finalizes through the witness-gated `consume_receipt`. Both core halves are beneficiary-agnostic. The built-in linear curve adds its own gate: `destroy` consumes the receipt only if the schedule has ended and `ctx.sender()` equals the beneficiary, a [deliberate measure](#) to keep the strand risk and the storage rebate with the party the teardown affects.

A transaction sender is always a keypair-controlled account and never an object address, so `ctx.sender() == beneficiary` cannot be satisfied when the beneficiary is an object. Because the `DestroyReceipt` is a hot potato that must be consumed in the same transaction and the linear curve exposes no other consumer, the failed gate reverts the whole teardown, including the `destroy_empty` call. A wallet whose beneficiary is an object can therefore never be torn down: the drained wallet object persists indefinitely and its storage rebate is permanently forfeited. No funds are at risk, since releases have already been paid out, but teardown is fully broken for a documented and supported configuration, and because the linear curve is the template that custom curves copy, the same gate propagates to any curve written against it.

The team is aware of this limitation and is evaluating remediation, including a capability-based teardown authority that decouples teardown from the beneficiary address.

Update: Resolved in [pull request #453](#). The team stated:

| *We implemented a capability-based approach to solving the issue.*

Conclusion

The vesting wallet is a curve-agnostic primitive for Sui Move that locks a single coin type for one beneficiary and releases it over time on a schedule, separating a wallet that custodies funds and enforces release accounting from a swappable curve module that defines how much has vested. OpenZeppelin audited the `vesting_wallet` primitive and the built-in `vesting_wallet_linear` curve, assessing the correctness of the release and conservation accounting, the type-level binding that ties each wallet to a single curve, the witness-gated attestation and teardown flow, and the transfer-to-object model used to fund wallets and pay beneficiaries. The example modules under `examples/vesting_wallet` were outside the scope of this engagement and were reviewed only as references for the intended integration patterns.

No issues of critical, high, or medium severity were identified within the audited scope. One low-severity issue and six notes were raised, concentrated in three areas: the precision and completeness of the documentation, consistency with established Move conventions, and the ergonomics of common integration patterns, such as wrapping a wallet or pointing its beneficiary at an object. Examples include coins transferred to a wrapped wallet's address remaining unclaimable until the wallet is unwrapped, an overstated description of the conditions under which the release path aborts, and the `Released` event omitting the payout coin id that an object beneficiary needs to correlate its pending receivings. The core guarantees of fund conservation, the cap that no release exceeds the funded balance, and the wallet-id binding that confines each attestation to the wallet it was minted for were not found to be affected. Separately, one issue concerning the teardown of wallets whose beneficiary is an object address was disclosed by the development team before the engagement and is documented in this report for completeness.

The codebase is cleanly structured, separating the curve-agnostic core from the built-in linear curve, and is accompanied by a test suite covering the in-scope modules. Because the primitive delegates schedule correctness entirely to the curve module that holds the witness and leaves the topology and beneficiary choices to the integrator, its effective security continues to rest on the honesty of curve modules and on correct integrator configuration, as set out in the trust assumptions. Addressing the surfaced items would tighten the documentation, align the code with Move conventions, and improve the ergonomics of these integration patterns, reducing the chance that an integrator adopts the primitive with a weaker guarantee than intended.

The OpenZeppelin development team is thanked for their responsiveness and willingness to provide technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.